

MAZE OF UCSC

Snake & Ladders Variant

README Documentation

Student: M.Y Hanan Mohamed

Course: SCS1301 Data Structures and Program Design in C

Professor: Dr. Manjusri Wickramasinghe

Status: Fully Implemented — Passes All Critical Test Cases

Contents

1	Overview	2
1.1	Core Features	2
2	Assumptions & Design Decisions	2
2.1	Bawana Effects	3
3	How to Compile	3
4	How to Run	3
4.1	Output includes:	4
5	Input File Format	4
5.1	seed.txt	4
5.2	stairs.txt	4
5.3	poles.txt	4
5.4	walls.txt	4
5.5	flag.txt	4
6	Special Game Logics & Handling	4
6.1	Infinite Loop Detection	4
6.2	Bawana Entrance Restriction	5
6.3	Flag Capture Mid-Movement	5
6.4	Wall Sanitization	5
6.5	Stair Direction Updates	5
6.6	Movement Bonuses & Consumables	5
6.7	Bawana Effects	5
6.8	Multiple Stairs/Poles at Same Cell	5
6.9	Flag Validation	5
7	Critical Test Cases Passed	6
8	Logging System (log.txt)	6
8.1	Example log entries:	6
9	Known Minor Deviations	7
10	Contact & Credits	7

1 Overview

“Maze of UCSC” is a 3-player, 3-floor turn-based maze game implemented in C. Players roll dice to move through a maze, use stairs/poles to teleport, avoid walls, survive Bawana effects, and race to capture the flag.

The game ends immediately when any player lands on the flag — even mid-movement.

1.1 Core Features

- Dice-based movement & direction control
- Dynamic stairs (direction changes every 5 rounds)
- One-way poles (slide down only)
- Wall obstacles
- Bawana area with randomized player effects
- Movement point (MP) system with bonuses & penalties
- Player capture mechanics
- Infinite loop detection & recovery
- Configurable via external files
- Comprehensive logging for debugging

2 Assumptions & Design Decisions

- **Bawana Interior:** 12 cells at [0,6-9,20-24] — 2 of each effect type + 4 random MP cells
- **Bawana Entrance:** [0,9,19] — only enterable when MP=0. Forces direction=NORTH
- **Starting Area:** [0,6-9,8-16] — players start here and must roll 6 to enter maze
- **Player Entry Cells:**
 - A: enters at [0,5,12] → direction NORTH
 - B: enters at [0,9,7] → direction WEST
 - C: enters at [0,9,17] → direction EAST
- **Captured players** reset to Player A’s start: [0,6,12], direction NORTH
- **Movement cost:** Each cell has consumable value 0-4 → deducted from MP
- **Movement bonuses:** 25% add 1-2 MP, 10% add 3-5 MP, 5% multiply MP by 2x or 3x (capped at 250)
- **Flag** must be placed on valid, reachable cell — validated via BFS at startup
- **Infinite loops** (stairs/poles) → detected → player reset to [0,6,12] with MP preserved
- **Walls** overlapping critical zones (spawn, Bawana, start) → automatically disabled & logged
- **Multiple stairs/poles** at same cell → resolved by Manhattan distance to flag → pole i stair priority
- **Flag capture** in intermediate step → game ends immediately → logged

2.1 Bawana Effects

- **Food poisoning:** Skip 3 turns → then placed randomly in Bawana → new effect applied
- **Disoriented:** Random direction for 4 turns
- **Triggered:** Move 2x rolled amount for 4 turns
- **Happy:** Instant +200 MP, no lasting effect
- **Random MP:** +10 to +100 MP, lasts 4 turns
- **MP 0** → player sent to random Bawana interior cell → effect applied

3 How to Compile

You need a C compiler (gcc recommended).

On Linux/macOS:

```
gcc -o maze main.c game.c -lm
```

On Windows (MinGW/MSYS2):

```
gcc -o maze.exe main.c game.c -lm
```

Note: -lm links math library (for `abs()` in `manhattan_distance`).

4 How to Run

Place these files in the same directory:

- `main.c`
- `game.c`
- `game.h`
- `seed.txt` (optional)
- `stairs.txt` (optional)
- `poles.txt` (optional)
- `walls.txt` (optional)
- `flag.txt` (optional)

Then run:

```
./maze          # Linux/macOS  
maze.exe       # Windows
```

Game runs automatically in console. Press Enter after each turn if prompted.

4.1 Output includes:

- Dice rolls
- Movement steps
- Stair/pole teleports
- Bawana effects
- Captures
- MP/direction status
- Win condition

5 Input File Format

All files are plain text — one entry per line.

5.1 seed.txt

Single integer (e.g., 12345) — used to seed `rand()`.

5.2 stairs.txt

Format: [start_floor, start_w, start_l, end_floor, end_w, end_l]

Example: [0,5,10,1,5,10]

5.3 poles.txt

Format: [start_floor, end_floor, w, l]

Example: [2,0,5,24] → slide from floor 2 to 0 at [5,24]

5.4 walls.txt

Format: [floor, start_w, start_l, end_w, end_l]

Example: [1,0,2,8,2] → horizontal wall on floor 1 from [0,2] to [8,2]

5.5 flag.txt

Format: [floor, w, l]

Example: [1,3,8]

Warning: Invalid or missing files → defaults loaded + warning logged.

6 Special Game Logics & Handling

6.1 Infinite Loop Detection

- Tracks last 100 positions during movement
- If revisit → reset player to [0,6,12] → MP preserved → direction NORTH
- Logged in log.txt: “LOOP DETECTION: Player X trapped...”

6.2 Bawana Entrance Restriction

- Only enterable when MP = 0
- If player tries to walk in with MP $\neq$ 0 $\rightarrow$ movement blocked
- When placed here (after effect) $\rightarrow$ forced direction = NORTH

6.3 Flag Capture Mid-Movement

- Game ends immediately if flag is captured at any step — even if more steps remain
- Logged: “FLAG CAPTURE OVERRIDE: Player X captured flag at step N...”

6.4 Wall Sanitization

- Any wall overlapping spawn/Bawana/start cells $\rightarrow$ automatically disabled
- Logged: “ASSUMPTION: Wall at [...] disabled — overlaps spawn cell”

6.5 Stair Direction Updates

- Every 5 rounds, stair direction_type randomly set to UP/DOWN/BIDIRECTIONAL
- Logged: “ASSUMPTION: Stair directions updated after 5 rounds.”

6.6 Movement Bonuses & Consumables

- Bonuses applied only once per cell $\rightarrow$ then cleared
- Consumable cost deducted even if movement blocked
- MP capped at 250

6.7 Bawana Effects

- Applied only once per landing
- Effects wear off after set turns (logged on recovery)
- Food poisoning recovery with MP=0 $\rightarrow$ re-transported to Bawana

6.8 Multiple Stairs/Poles at Same Cell

- Priority: Poles $\neq$ Stairs
- Tie-breaker: Manhattan distance to flag
- First encountered wins on distance tie (not random — for determinism)

6.9 Flag Validation

- BFS from all 3 entry points $\rightarrow$ ensures flag is reachable
- If unreachable $\rightarrow$ replaced with random valid reachable cell $\rightarrow$ warning logged

7 Critical Test Cases Passed

All 7 test cases handled robustly:

1. **Test Case 1: Walls on Spawn Cells**
→ Walls automatically disabled → players enter normally.
2. **Test Case 2: Walls Inside Bawana**
→ Walls removed → Bawana remains fully accessible.
3. **Test Case 3: Infinite Stair Loop**
→ Loop detected → player reset → MP preserved → logged.
4. **Test Case 4: Stair-Pole Infinite Loop**
→ Loop detected → player reset → MP preserved → logged.
5. **Test Case 5: Flag in Unreachable Location**
→ BFS fails → flag replaced → warning logged.
6. **Test Case 6: Overlapping Stairs and Poles**
→ Pole priority respected → distance-based selection → logged.
7. **Test Case 7: Maximum Stair Density (2 per cell)**
→ Distance-based selection → direction permissions respected → logged.

8 Logging System (log.txt)

Automatic logging to `log.txt` for:

- Assumptions made (e.g., wall sanitization, stair direction updates)
- Warnings (e.g., invalid flag, duplicate stairs/poles)
- Infinite loop detection & recovery
- Flag capture override
- Bawana effect applications
- File loading fallbacks
- Pathfinding results

8.1 Example log entries:

```
ASSUMPTION: Wall at [0,5,12]-[5,12] disabled overlaps spawn cell
WARNING: Flag in flag.txt at [1,5,5] is unreachable, replacing...
LOOP DETECTION: Player B trapped in infinite loop at [1,5,10]...
FLAG CAPTURE OVERRIDE: Player A captured flag at step 2...
```

9 Known Minor Deviations

- **Player reset position:** All players reset to [0,6,12] (Player A's start) — not their original start. (Easy to modify if needed.)
- **Tie-breaking for stairs:** Uses “first encountered” instead of random — for deterministic replayability.
- **Round limit:** No max round limit — game ends only on flag capture. (Add MAX_ROUNDS if needed for safety.)
- **Bawana effect duration:** Happy effect has no turn counter (as per design — it's instant).

10 Contact & Credits

Author: M.Y Hanan Mohamed
Course: SCS1301 Data Structures and Program Design in C
Instructor: Dr. Manjusri Wickramasinghe
Institution: University of Colombo School of Computing